

Лабораторна робота №6

Вказівники. Динамічний розподіл пам'яті

Варіант - 14

Павліка Максим Олександрович

Завдання:

Оформити програми, написані для масивів та стрічок у л.р. 3 - 5 використовуючи динамічне виділення пам'яті (calloc, malloc, free) та два способи доступу

1. через []. Тобто для одновимірного масиву цілих елементів

A[i]

2. через оператор розіменування *. Тобто так

*(A+i*len),

де len=sizeof(int).

Код програми:

Lab(3.1_1):

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    int lengthOfA;
    int counterForB = 0;

    printf("Enter length of A: ");
    scanf("%d", &lengthOfA);
```

```

    if (lengthOfA <= 0) return 1;

    int *A = (int *)malloc(lengthOfA * sizeof(int));
    int *B = (int *)malloc(lengthOfA * sizeof(int));

    printf("Enter %d numbers: ", lengthOfA);
    for (int i = 0; i < lengthOfA; i++) {
        scanf("%d", &A[i]);
    }

    for (int i = 0; i < lengthOfA; i++) {
        if (A[i] < 0) {
            B[counterForB] = A[i] * A[i];
            counterForB++;
        }
    }

    printf("Negative numbers from A, squared in B: \n");
    for (int i = 0; i < counterForB; i++) {
        printf("%d ", B[i]);
    }
    printf("\n");

    free(A);
    free(B);
    return 0;
}

```

Lab(3.1_2):

```
#include <stdio.h>

#include <stdlib.h>

int main() {

    int lengthOfA;

    int counterForB = 0;

    int len = sizeof(int);

    printf("Enter length of A: ");

    scanf("%d", &lengthOfA);

    if (lengthOfA <= 0) return 1;

    int *A = (int *)malloc(lengthOfA * sizeof(int));

    int *B = (int *)malloc(lengthOfA * sizeof(int));

    printf("Enter %d numbers: ", lengthOfA);

    for (int i = 0; i < lengthOfA; i++) {

        scanf("%d", (A + i));

    }

    for (int i = 0; i < lengthOfA; i++) {

        if (*(A + i) < 0) {

            *(B + counterForB) = (*(A + i)) * (*(A + i));

            counterForB++;

        }

    }

    printf("Negative numbers from A, squared in B: \n");

    for (int i = 0; i < counterForB; i++) {
```

```

        printf("%d ", *(B + i));
    }
    printf("\n");

    free(A);
    free(B);
    return 0;
}

```

Lab(3.2_1):

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main() {
    int i_A, j_A;

    printf("Enter i-rows for A: ");
    scanf("%d", &i_A);
    printf("Enter j-columns for A: ");
    scanf("%d", &j_A);

    if (i_A <= 0 || j_A <= 0) return 1;

    int *A = (int *)malloc(i_A * j_A * sizeof(int));

    int elements = 0;
    double result = 1;

    printf("Enter matrix elements: \n");
    for(int i = 0; i < i_A; i++) {

```

```

        for(int j = 0; j < j_A; j++) {
            scanf("%d", &A[i * j_A + j]);
        }
    }

    for(int i = 0; i < i_A; i += 2) {
        for(int j = 0; j < j_A; j += 2) {
            if (A[i * j_A + j] > 0) {
                result *= A[i * j_A + j];
                elements++;
            }
        }
    }

    if (elements > 0)
        printf("Seredne geometricne %lf\n", pow(result, 1.0 /
elements));

    free(A);

    return 0;
}

```

Lab(3.2_2):

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main() {
    int i_A, j_A;

    printf("Enter rows and columns: ");
    scanf("%d %d", &i_A, &j_A);

```

```

if (i_A <= 0 || j_A <= 0) return 1;

int *A = (int *)malloc(i_A * j_A * sizeof(int));

int elements = 0;
double result = 1;

printf("Enter matrix elements: \n");
for(int i = 0; i < i_A; i++) {
    for(int j = 0; j < j_A; j++) {
        scanf("%d", (A + i * j_A + j));
    }
}

for(int i = 0; i < i_A; i += 2) {
    for(int j = 0; j < j_A; j += 2) {
        int val = *(A + i * j_A + j);
        if (val > 0) {
            result *= val;
            elements++;
        }
    }
}

if (elements > 0)
    printf("Seredne geometricne %lf\n", pow(result, 1.0 /
elements));

free(A);
return 0;
}

```

Lab(4_1):

```
#include <stdio.h>

#include <stdlib.h>

int main() {

    FILE *f = fopen("input.txt", "r");

    if (!f) return 1;

    int n;

    fscanf(f, "%d", &n);

    int (*matrix)[n] = malloc(n * sizeof(*matrix));

    if (!matrix) {

        fclose(f);

        return 1;

    }

    for (int i = 0; i < n; i++) {

        for (int j = 0; j < n; j++) {

            fscanf(f, "%d", &matrix[i][j]);

        }

    }

    fclose(f);

    int sP, lP, sR = -1, lR = -1;

    for (int i = 0; i < n; i++) {

        for (int j = 0; j < n; j++) {

            int num = matrix[i][j];

            int isP = (num > 1);
```

```

        for (int d = 2; d * d <= num; d++) {
            if (num % d == 0) {
                isP = 0;
                break;
            }
        }

        if (isP) {
            if (sR == -1 || num < sP) { sP = num; sR = i; }
            if (lR == -1 || num > lP) { lP = num; lR = i; }
        }
    }

    if (sR != -1 && lR != -1 && sR != lR) {
        for (int j = 0; j < n; j++) {
            int t = matrix[sR][j];
            matrix[sR][j] = matrix[lR][j];
            matrix[lR][j] = t;
        }
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    free(matrix);

```

```
    return 0;
}
```

Lab(4_2):

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *f = fopen("input.txt", "r");
    if (!f) return 1;

    int n;
    fscanf(f, "%d", &n);

    int *matrix = (int *)malloc(n * n * sizeof(int));

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            fscanf(f, "%d", (matrix + i * n + j));
    fclose(f);

    int sP, lP, sR = -1, lR = -1;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int num = *(matrix + i * n + j), isP = (num > 1);
            for (int d = 2; d * d <= num; d++) if (num % d == 0)
                isP = 0;

            if (isP) {
                if (sR == -1 || num < sP) { sP = num; sR = i; }
                if (lR == -1 || num > lP) { lP = num; lR = i; }
            }
        }
    }
}
```

```

        }

    }

}

if (sR != -1 && sR != lR) {
    for (int j = 0; j < n; j++) {
        int t = *(matrix + sR * n + j);
        *(matrix + sR * n + j) = *(matrix + lR * n + j);
        *(matrix + lR * n + j) = t;
    }
}

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) printf("%d ", *(matrix + i * n
+ j));
    printf("\n");
}

free(matrix);
return 0;
}

```

Lab(5_1):

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main() {
    char *text = (char *)calloc(200, sizeof(char));
    printf("Enter text: ");
    fgets(text, 200, stdin);

    char (*words)[6] = calloc(100, sizeof(*words));

```

```

int wCount = 0, lIdx = 0;
for (int i = 0; text[i] != '\0' && text[i] != '\n'; i++) {
    if (text[i] >= 'a' && text[i] <= 'z') {
        words[wCount][lIdx++] = text[i];
    } else if (text[i] == ',' || text[i] == '.') {
        words[wCount++][lIdx] = '\0';
        lIdx = 0;
        if (text[i] == '.') break;
    }
}

for (int w = 0; w < wCount; w++) {
    for (int i = strlen(words[w]) - 1; i >= 0; i--) {
        printf("%c", words[w][i]);
    }
    printf(w < wCount - 1 ? "," : ".\n");
}

free(words);
free(text);

return 0;
}

```

Lab(5_2):

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main() {

```

```

char *text = (char *)calloc(200, sizeof(char));
printf("Enter text: ");
fgets(text, 200, stdin);

char *words = (char *)calloc(100 * 6, sizeof(char));

int wCount = 0, lIdx = 0;
for (int i = 0; *(text + i) != '\0' && *(text + i) != '\n';
i++) {
    if (*(text + i) >= 'a' && *(text + i) <= 'z') {
        *(words + wCount * 6 + lIdx++) = *(text + i);
    } else if (*(text + i) == ',' || *(text + i) == '.') {
        *(words + wCount * 6 + lIdx) = '\0';
        wCount++;
        lIdx = 0;
        if (*(text + i) == '.') break;
    }
}

printf("Reversed: \n");
for (int w = 0; w < wCount; w++) {
    int len = strlen(words + w * 6);
    for (int i = len - 1; i >= 0; i--) printf("%c", *(words +
w * 6 + i));
    printf(w < wCount - 1 ? ", " : ".\n");
}

free(words); free(text);
return 0;
}

```

Скріншоти виконання з різними наборами вхідних даних:

P.S *(Код виконується так само, як і в оригінальних файлах лаб. 3-5)*

